

Clustering

Rui

```
r format(Sys.Date(), '%B %d, %Y')
```

目录

1	K-Means	1
1.1	Computation	1
1.2	Visualization	2
2	Hierachical Clustering	2
3	No. of Clusters	3
3.1	Within-Cluster SS	3
3.2	Silhouette Method	3
3.3	Majority Rule	3
4	Two Step Clustering	4
5	Clustering for Cases and Variables	4
5.1	Q1	4
5.2	Q2	5

```
{r results='hide',message=FALSE} library("parameters") #for clustering summary library("see")
library("factoextra") #for clustering plotting library(bruceR) library(tidyverse)

{r} athlete = import("data13-02.xlsx") %>% select(-1)
```

1 K-Means

1.1 Computation

- **x**: numeric matrix-like data
- **centers**: either the number of clusters, say `kk`, or a set of initial (distinct) cluster centres. If a number, a random set of (distinct) rows in `x` is chosen as the initial centres.

- `iter.max`: the maximum number of iterations allowed.
- `nstart`: if centers is a number, how many random sets should be chosen. Trying several random starts is often recommended.

```
{r} kmean_results = kmeans(athlete, centers = 4, iter.max = 10,
nstart = 1)
```

`kmeans` returns an object of class “`kmeans`” which has a `print` and a `fitted` method. It is a list with at least the following components:

- `cluster` : A vector of integers (from 1:k) indicating the cluster to which each point is allocated.
- `centers`: A matrix of cluster centres.
- `totss`: The total sum of squares.
- `withinss`: Vector of within-cluster sum of squares, one component per cluster.
- `tot.withinss`: Total within-cluster sum of squares, i.e. `sum(withinss)`.
- `betweenss`: The between-cluster sum of squares, i.e. `totss-tot.withinss`.
- `size`: The number of points in each cluster.

```
{r} kmean_results$cluster #group labels for each observation
```

```
{r} kmean_results$centers #centers
```

```
{r results='hold'} kmean_results$totss # total variance kmean_results$withinss #
within-group variance kmean_results$tot.withinss # sum of within SS kmean_results$betweenss
#between-group variance
```

```
{r} kmean_results$size # how many cases in each cluster
```

1.2 Visualization

Use `factoextra::fviz_cluster(object, data)`. For more details about useful parameters, see the help document. Remember that `object` and `data` are two most indispensable parameters.

```
{r} fviz_cluster(kmean_results, data = athlete)
```

One way to start is from random centers; another way is to specify a few sets of centers and start from them.

Clearly, starting from different centers, either random or specified, the results may vary, though not significant. If the iteration will converge eventually, then given sufficiently-large iteration time, the center will be determined and fixed.

```
{r} initial_centers = import("data13-02a.xlsx") %>% select(-1) kmean_results_initialized
= kmeans(athlete, initial_centers , iter.max = 10,
nstart = 1) fviz_cluster(kmean_results_initialized, data = athlete)
```

2 Hierarchical Clustering

```
{r} automobile = import("data13-01.xlsx") %>% select(c('type','price','engine_s','horsepow',
'wheelbas','width','length','curb_wgt',      'fuel_cap','mpg')) %>% drop_na() %>%
scale()
```

Use “`hclust()`” from base R and transmit a **dissimilarity** structure produced by “`dist()`”.

```
{r} distMat = dist(automobile, method = "euclidean") # Euclidean distance matrix hc =
hclust(distMat) # hierarchical clustering, using the default method
```

Use “`plot(hclust_obj)`” to visualize the hierarchical clustering. Note that you should specify “`hang = -1`” to ensure all nodes touch the same horizontal line and looks decent. If the sample isn’t that large, set “`labels = TRUE`” (by default) to see the corresponding observations.

```
{r} plot(hc,labels=FALSE,hang = -1) plot(hc,labels=FALSE)
```

Moreover, you can draw dendrogram with grouping.

```
{r warning=FALSE} fviz_dend(hc,k=3, rect = T)
```

Also, you can also check how many members for each cluster and make a frequency table.

Note that the grouping might change if we change cluster number. And this in essence is the pros of hierarchical clustering.

```
{r} memberLabels = cutree(hc,3) table(memberLabels)
```

Then, we can get the centers of cluters by “`mean`” summary statistics. Use “`aggregate()`” to achieve that.

- `x`: usually a data frame.
- `by`: a list of grouping elements, each **as long** as the variables in the data frame `x`. The elements are coerced to factors before use.
- `FUN`: a function to compute the summary statistics which can be applied to all data subsets.

```
{r} aggregate(automobile,list(memberLabels),mean)
```

Moreover, we can use Silhouette plot to check whether some cases are outliers.

```
{r} library(cluster) plot(silhouette(cutree(hc,3), distMat))
```

3 No. of Clusters

3.1 Within-Cluster SS

Looking for the “elbow” where the SS starts to slow down its drop.

```
{r} fviz_nbclust(automobile, kmeans, method = "wss", print.summary=TRUE)
```

here suggests 2 or 3.

3.2 Silhouette Method

Looking for the value that sets the highest average silhouette width, which indicates the optimal number of clusters.

```
{r} fviz_nbclust(automobile, kmeans, method = "silhouette", print.summary=TRUE)
```

3.3 Majority Rule

`n_clusters()` is the main function to find out the optimal numbers of clusters present in the data based on the maximum consensus of a large number of methods.

Essentially, there exist many methods to determine the optimal number of clusters, each with pros and cons, benefits and limitations. The main `n_clusters()` function proposes to run all of them, and find out the number of clusters that is suggested by the **majority** of methods (in case of ties, it will select the most parsimonious solution with fewer clusters).

Note that we also implement some specific, commonly used methods, like the Elbow or the Gap method, with their own visualization functionalities.

Also note that the `n_clusters()` function will do the normalization automatically (by default).

```
{r results='asis'} n_clusters(data.frame(automobile)) plot(n_clusters(data.frame(automobile)))
```

4 Two Step Clustering

Some people prefer using hierarchical methods first to obtain the number of clusters and their centroids. Then use these as inputs to run non-hierarchical analysis. This approach is called Two-Step Clustering.

```
{r eval=FALSE} # two step clustering in R, not required library(prcr) d <- pisaUSA15 m3
<- create_profiles_cluster(d,                                broad_interest, enjoyment,
instrumental_mot, self_efficacy,                             n_profiles = 3) summary(m3)
```

5 Clustering for Cases and Variables

Infrastructure construction includes various dimensions such as transportation, postal and telecommunications, water supply and power supply. “construction.xlsx” records the development level of 16 infrastructure

construction aspects in 7 countries. Please answer the following questions:

1. Please use hierarchical methods to determine how many different types of infrastructure construction are included in these 16 indicators, and briefly explain the possible meanings of each type.
2. Calculate the average value of each type of indicator for each country in the previous question as the overall measure of infrastructure construction for that type (assuming this operation is reasonable). Use the k-means method to cluster the 7 countries based on their different overall indicators, distinguish between developed and developing countries, and determine which countries belong to each category.

5.1 Q1

```
{r results='hide',message=FALSE} library(factoextra) library(showtext) showtext_auto()

{r message=FALSE,warning=FALSE} cons = import("construction.xlsx") rownames(cons) =
cons[,1] cons = cons %>% select(-1) attach(cons)

{r} distance = dist(cons) hc = hclust(distance)

{r warning=FALSE,message=FALSE} plot(hc,hang = -1)
```

From the cluster dendrogram above, it's suggested to classify all 16 indexes into 3 grand categories. From indexes contained in each category, we can infer that the three dimensions are:

- Medical resources
- Education level
- Traffic capacity

5.2 Q2

```
{r} grouplabel = cutree(hc,k = 3)

{r} km_country = aggregate(cons,list(grouplabel),mean) %>% select(-1) %>% t() %>%
kmeans(centers = 2) country_label = km_country$cluster country_label
```

From the *k*-means method, we can see that the seven countries are classified into two grand categories, developed and developing countries, where the first group consists of D, E & F, and all else as the second group. However, clustering analysis doesn't directly tell us the attributes of two groups, therefore, we have to "label" the groups by checking the data.

Note that here D, E & F are labeled as group "2".

```
{r results='asis'} cons %>% t() %>% as.data.frame() %>% mutate( develop = country_label
) %>% group_by(develop) %>% summarise_all(mean) %>% stargazer::stargazer(header =
```

```
FALSE,                                title = 'Index Comparison Between Groups',      flip
= T,summary = FALSE)
```

From the summarization table, apparently that among all the 16 indexes, group 1 outbeats group 2 (i.e., D, E & F). Therefore, we can say

- Developing countries: D, E, F;
- Developed countries: A, B, C, G.

```
{r} fviz_cluster(km_country,          data = aggregate(cons,list(grouplabel),mean)
%>%                                select(-1) %>% t())
```

All done. Finally, {r} `detach(cons)`